

GIT AND THE TEAM GATE

WEEK 1, DAY 2

Git and code review, deepened. Foundation layer.



Yesterday

ruff was happy. mypy was happy.
And the total came back as 35.349999999999999994.

Today

GIT WAS HAPPY.

THE SPINE OF THE DAY

COMMIT, BRANCH, MERGE, REVIEW

FOUR WORDS, FOUR JOBS

- | | | |
|----------|-----------------|---|
| 1 | a commit | A claim about a change. |
| 2 | a branch | A claim about a state. One pointer, forty characters. |
| 3 | a merge | Git agreeing two texts can coexist. Nothing more. |
| 4 | a review | A person deciding whether the change is right. |

Only one of those four asks whether the code is correct.

A COMMIT IS A SNAPSHOT, NOT A DIFF

IDEA ONE OF THREE

WHAT A COMMIT STORES

- A complete snapshot of the tree.
- A pointer to its parent, or parents.
- A message.
- Git computes the diff on demand, by comparing snapshots.

WHICH IS WHY

Checking out an old commit is fast. It is not replaying history. It is reading a snapshot.

Commits are immutable. They are identified by a hash of their own content.

THE MESSAGE IS THE ONLY PLACE THE WHY LIVES

COMMIT HYGIENE

A COMMIT MESSAGE THAT WORKS

```
Add failing test for ledger.largest
```

```
The report needs the largest entry to flag  
for review. No test covered this, so the  
behaviour was never pinned down.
```

SUBJECT

says WHAT

Body says WHY

The diff already tells a reviewer the what. The body is the only part carrying information the diff does not.

If you cannot say why, you may be committing something you do not need.

HISTORY IS A GRAPH, A BRANCH IS A POINTER

IDEA TWO OF THREE

THE MODEL

```
A ---- B ---- C      <- main
      \
      D ---- E        <- feature
```

common ancestor of C and E is B
git compares B..C and B..E

A BRANCH IS

a file containing one commit hash.

That is all it is.

Creating a branch is instant. Deleting one deletes no commits.

YOU CANNOT LOSE A COMMIT YOU MADE

ONLY THE LABEL

Almost nothing you do destroys a commit. It moves pointers.

FOUR THINGS PEOPLE FEAR, AND WHY THEY SHOULD NOT

- Delete a branch: you threw away the label, not the ten commits.
- Rebase: git wrote new commits. The old ones are still there.
- reset --hard: the pointer moved. The commit did not.
- git reflog shows every position HEAD has held.

A MERGE ASKS A QUESTION ABOUT TEXT

IDEA THREE OF THREE

DIFFERENT LINES

Git takes both.
No conflict.
No questions asked.

SAME LINES

Git cannot choose.
It stops and
asks you.

Whether the result makes sense is not a question git is capable of asking.



THE SENTENCE THAT MATTERS

**DIFFERENT LINES MEANS
GIT TAKES BOTH, SILENTLY.**

It has no idea what a function is, what a name refers to, or what your code means.

DEMO-1 BACKUP: GIT REPORTED A CLEAN MERGE

THE SEMANTIC CONFLICT

GIT SAYS

```
$ git merge add-audit
Auto-merging tests/test_ledger.py
Merge made by the 'ort' strategy.
```

```
$ git status --short
$
```

PYTEST SAYS

```
ImportError: cannot import
name 'find_reference'
from 'ledger'
```

```
src/audit.py:5
1 failed, 2 passed
```

Two correct commits. One broken result. No line of code in the repository is the bug.

A reviewer of either pull request in isolation approves it, correctly.

POINT AT THE ONE THAT BROKE IT

NINE PLAUSIBLE MESSAGES

GIT LOG --ONELINE

```
Add is_empty helper
Add test for entry_count
Document entry_count
Add largest helper
Simplify total by accumulating in a loop
Expand module docstring
Add references helper
Add entry_count helper
Add ledger with exact Decimal totals and a test
```

WHAT WE KNOW

The suite is red today.

It was green at the first commit.

So exactly one commit changed the answer.

Reading diffs newest-first is linear search by eyeball.

EIGHT CANDIDATES. TWO TEST RUNS

GIT BISECT

8

CANDIDATE COMMITS

2

TEST RUNS

1

ANSWER

THE WHOLE SEARCH

```
➤ git bisect start
$ git bisect bad
$ git bisect good <first commit>
$ git bisect run python3 -m pytest -q

... is the first bad commit
    Simplify total by accumulating in a loop
```

WHY THE COMMIT DISCIPLINE PAYS

WHAT BISECT NEEDS FROM YOU

THREE PREREQUISITES

- 1 atomic** One commit, one reason. Bisect can point at something specific.
- 2 a test** A failing test is what bisect asks. Without one, nothing to search on.
- 3 a message** The why, for whoever judges this commit six months from now.

Commit hygiene is not tidiness. It is what makes the search work.

The commit bisect found had the most innocent message in the list.

REVIEW IS A DECISION

NOT A LIST OF OBSERVATIONS

WOULD YOU PUT YOUR NAME NEXT TO THIS CHANGE?

Not: do I like it. Not: would I have written it this way. Not: are the names good.

A lot of people find a real defect and approve anyway. Notice yourself doing it.

RUN THE GATE BEFORE YOU READ A LINE

YOU ARE NOT A LINTER

YESTERDAY'S THREE COMMANDS

```
ruff check .  
mypy src tests  
pytest -q
```

If any is red,
send it back.

WHICH CHANGES THE MEASURE OF A GOOD REVIEW

- Every finding the machine hands you is attention you did not have to spend.
- The useful measure is not how many things you found.
- It is how many things you found that a machine could not have.

THE READING ORDER

NOT THE ORDER THE DIFF GIVES YOU

- | | | |
|---|--------------------|--|
| 1 | commit messages | What did the author think they were doing |
| 2 | the file list | Is every file here part of this change |
| 3 | the tests | Would they fail if the code were wrong |
| 4 | the implementation | Now, and only now |
| 5 | what is missing | A diff cannot show you the test nobody wrote |

A diff shows the files the author wanted you to look at. The file list shows the shape.

HOW TO WRITE A FINDING

ASK, DO NOT INSTRUCT

FINDS THE BUG, LEAVES THE REASONING

"What happens at exactly 500.00?"

GETS THE LINE CHANGED, TEACHES NOTHING

"Change this to >="

FOUR FIELDS, EVERY FINDING

WHERE	file and line
WHAT	one sentence
CLASS	MACHINE (a gate should have caught it) or HUMAN
ASK	what you want the author to do

Separate blocking from optional, or the author will guess which parts matter.

PRACTICAL: REVIEW THE PULL REQUEST

FORTY MINUTES, ALONE, NO AI

THREE STEPS, IN THIS ORDER

- 1 gate** All three, before you read a line. Every red is attention saved.
- 2 read** Commits, file list, tests, code, what is missing.
- 3 decide** Write REVIEW.md. Class every finding. Then give a verdict.

All MACHINE findings means a script did your job. No MACHINE findings means you never ran the gate.

THE PASS BAR

ALL FIVE, AND APPROVE IS A FAIL

- 1 At least one MACHINE finding, with the gate output referenced
- 2 The boundary bug found, and classified HUMAN
- 3 At least one of: the tautological test, the missing boundary test, the swept-in change
- 4 Every finding carries a class
- 5 Verdict of request changes, with a reason naming a HUMAN finding

A verdict of approve fails, however many findings you listed.

A GREEN TICK IS NOT EVIDENCE

A green tick is evidence that a specific set of checks passed.
It is not evidence that the change is right.
Nobody has ever built a tool that produces the second thing.

Tomorrow the author of the pull request is a model. The reading order does not change.

TOMORROW: THE ENGINEERING BAR FOR THE AI LAYER.